ADRL implementation plan

27 October 2026

Build the guarded Claude Code path first, then prove it before adding Codex.

Deliver seven reviewable changes around one core process and one SQLite database. Keep Jev, truthful memory, the evidence graph and all nine taxonomy buckets. Remove redundant machinery; do not build a new framework or platform.

This document is a plan, not implementation approval. No repository edits, tests or paid inference were performed. The reviewed repository main was 20ca99d.

Milestones

Milestone	What must be demonstrated
After PR 2	Safe transport with one continuity and execution owner; every outbound path remains constrained.
After PR 4	First mocked end-to-end Jev and ledger flow. This does not establish that paid Jev integration works.
After PR 5	The full retained capability: advisor, authoritative ledger, background enrichment and useful graph queries.
After PR 6	A measured Claude Code release, subject to explicit approval and declared quality, latency and cost limits.
After PR 7	Codex portability demonstrated independently, without copying routing policy or state machines.

Decision needed

Authorize repository implementation and select the execution environment to start mocked development. Provider access, data scope, spend and release tolerances must be approved separately before paid or live work.

Sequence: PR 1 → PR 2 → PR 3 → PRs 4 and 5 → PR 6 → PR 7. PR 5 can proceed alongside PR 4 once PR 3 is complete.

Core design and operating rules

The response path stays small. The harness adapter handles protocol and identity; the core owns routing, constraints and execution. SQLite is the authoritative record. Optional embeddings and graph projections run in the background.

1 Thin harness adapter

Decode and encode native protocol messages without adding routing policy. Start with Claude Code and add Codex only after the first slice is proven.

2 Continuity owner

Retain the selected route through tool results and streaming. Request a new decision only at an explicit supported boundary.

3 Single Jev advisor

Use one typed, bounded call for a new decision. Return exactly one authoritative choice or abstain. Laya can later replace this advisor; it is not a chained second opinion.

4 Constrained executor

Check permissions, feasibility and budget before every outbound attempt. Own intended, attempted and actually served model transitions.

5 Authoritative ledger

Record the decision, attempts, outcome and verifier evidence. Preserve unknown outcomes and late evidence without changing historical facts.

6 Background evidence map

Build embeddings and graph projections from allowed evidence. Explain routes, repeated failures and proposed changes; never let the graph become routing authority.

Rules that apply even during failure

- Unrecognized traffic preserves native semantics only when its original destination remains permitted. Otherwise stop.
- Advisor abstention or timeout keeps a feasible permitted route or configured permitted native fallback. No feasible target means a visible refusal.
- Check context permissions before constructing advisor payloads. Apply the same limits to provider internal fallback and retries.
- Unknown quality stays unknown. Infrastructure or dialect failure is not automatically task difficulty. No automatic graph promotion, permanent utility exclusion or difficulty pre-gate.
- Bypass disables optimization, not safety. Rollback must not restore the known unsafe fail open behavior.

Build safe transport and continuity

PR 1 Make every exit path obey the same constraints

Primary ADRL-SAF-001; secondary SAF-002/003/004/005/006, FND-004, OPS-008. Depends on: none.

Existing files: src/router/policy.py, privacy.py, live_router.py; src/proxy/capture_proxy.py; config/litellm-local.yaml; docs/adr-index.md.

Changes: reuse one feasibility and destination check across planning, errors, fallbacks and advisor preparation. Reject unavailable permissions rather than passing unconditionally to Anthropic. Disable provider fallbacks that widen destinations; allow compatible native passthrough only under the same constraints.

Align ADR rows in this PR: FND-004 conditional fail-open; RTG-003/006 single-advisor direction; SEM-004/005 protocol-backed eligibility and boundaries, without a utility blacklist. Preserve IDs and history or supersede through the index; do not claim evidence prematurely.

Acceptance: pinned session plus planner/scanner failure sends zero cloud requests; prohibited advisor call is rejected before payload construction; no feasible candidate means a controlled stop; fallback never expands permissions; kill switch preserves restrictions.

Rollback: disable new optimization while retaining these safety fixes.

Key tests: tests/test_privacy.py, test_hook_feasibility.py, test_hook_failopen.py, test_router_proxy.py.

PR 2 Give Claude Code one continuity and execution owner

Primary ADRL-CAS-003; secondary SEM-001/002/003/005, CAS-004/005/006/007. Depends on: PR 1.

Existing files: src/proxy/capture_proxy.py; src/router/live_router.py, discriminator.py, episode.py, state.py, escalation_controller.py, escalate.py.

Proposed new files, only where extraction genuinely simplifies ownership: src/router/adapters/anthropic.py and src/router/execution.py.

Changes: separate protocol handling from routing judgment. Give interactions stable identity and continuity ownership; never merge unrelated anonymous sessions. Preserve native tools, IDs, ordering, cancellation and streaming. Execution owns intended, attempted and served models. Reuse routes without fresh advice; unknown boundaries never cause speculative switches.

Allow bounded permitted retries before response commitment. After commitment, surface interruption and wait for a safe boundary; never silently replay tool-use content. Remove duplicate handoff mutation paths only after moving their behavior tests.

Acceptance: a user request plus multiple tool results causes one decision; concurrent sessions cannot borrow state; partial tool stream plus disconnect never causes a silent second-model replay; failed fallback does not become the served model; cancellation and unknown protocol preserve contract.

Rollback: adapter/optimization bypass to the guarded native path; preserve ledger records.

Key tests: tests/test_live_router.py, test_router_proxy.py, test_state.py, test_escalation_controller.py; existing tests/fixtures/handshakes/.

Build truthful memory and bounded advice

PR 3 Make evidence truthful before making it clever

Primary ADRL-MEM-001; secondary MEM-002/003/004/005/009, OPS-006. Depends on: PRs 1-2.

Existing files: src/router/memory_ports.py, memory_sqlite.py, memory_facade.py, outcomes.py, live_verification.py, telemetry.py.

Changes: record stable route, attempt and event IDs. Keep recommendation, attempts and served response distinct. Preserve pending → closed_turn → closed_final and late evidence. Store verified success, failure or unknown; verifier provenance, cause, policy/advisor/harness/model versions and usage. Cloud success does not prove local sufficiency.

Use additive migrations with backup, version checks and old-row compatibility. Preserve historical labels and frozen reports. Interrupted requests are incomplete. Expose recorder degradation; unrecorded traffic cannot support evaluation or promotion.

Simplify storage to SQLite or an explicit disabled state; remove ambiguous False/None fallback chains.

Acceptance: duplicate events are idempotent; late verification attaches to the exact route; interrupted/fallback paths preserve actual facts; migration and reopen preserve old rows; full/locked database produces an explicit degraded state; secrets/raw prompts are not persisted.

Rollback: retain additive schema and old rows; disable optimization on incompatible state rather than down-migrate destructively.

Key tests: tests/test_memory_contract.py, test_memory_sqlite.py, test_memory_facade.py, test_outcomes.py, test_live_verification.py.

PR 4 Replace semantic routing machinery with one bounded Jev advisor

Primary ADRL-RTG-006; secondary RTG-002/003/005/008, SAF-001, LRN-004/005/006. Depends on: PRs 1-3.

Existing files: src/router/llm_classifier.py, features.py, policy.py, backends.py, eval_classifier.py, live_router.py; config/backends.yaml.

Proposed new files: src/router/advisor.py and src/router/jev_advisor.py.

Changes: one typed contract returns exactly one authoritative permitted choice or abstention. Choose candidate or rung, not both; resolve endpoints separately. Check context permissions before payload construction. Share the parser across runtime and evaluation. Bound asynchronous I/O with one total deadline for connection, response, SDK and provider internal retries. Record advisor cost, latency and versions.

Retire replaced lexicons, synthetic scores, duplicate tier/frontier answers, JSON repair and middle-only dispatch after caller coverage. Keep features needed for protocol, constraints and telemetry. Defer Laya; add no plugin framework.

Acceptance: mocked provider tests cover valid and unknown choices, malformed responses, timeout, rate limits, cancellation and abstention. Every result permits a route or controlled stop. Continuations skip redundant advice. Stalled calls do not block other streams; all retries respect the total deadline. This is not evidence from paid Jev traffic.

Rollback: disable Jev and use the guarded native baseline, not the deleted semantic stack.

Restore evidence access and prove the slice

PR 5 Keep memory enrichment and the graph off the response path

Primary ADRL-MEM-007; secondary MEM-001/005/006/008, OPS-001/006. Depends on: PR 3; can progress alongside PR 4.

Existing files: src/router/memory_facade.py, memory_backfill.py, memory_sqlite.py, graph_lite.py, insights.py.

Changes: append essential evidence before optional enrichment. Move embedding calls off synchronous planning. Use a bounded in-process worker and SQLite checkpoint, with one projection writer and deterministic, idempotent replay. Do not retain raw prompts for replay; optional non-recoverable embeddings stay missing.

Deliver three CLI/query contracts: explain a route; find repeated verified failures; trace evidence for a proposed change. Cite underlying events and distinguish observation from inference. Keep useful neighbor and multi-hop traversal; defer unrecorded ancestry. The graph has no routing authority.

Acceptance: slow/offline embedder and stale/missing graph do not delay advice/dispatch; crash/restart resumes projection without duplicate edges; rebuilt queries reproduce recorded facts; private turns produce no prohibited embedding/hash. SQLite lock contention stays within the declared response budget.

Rollback: stop enrichment and rebuild derived tables; never delete authoritative ledger events.

PR 6 Prove the Claude Code slice and remove obsolete paths

Primary ADRL-EVL-006; secondary EVL-001/002/003/004/005/007/008, LRN-001/002/007, OPS-004/008. Depends on: PRs 1-5.

Existing files: src/router/counterfactual.py, verifier.py, learning_contract.py, learning_readiness.py; tests/test_counterfactual.py; docs/adr-index.md, README.md; .github/workflows/test.yml.

Proposed new report: reports/jev-claude-pilot.md.

Changes: freeze sanitized tasks, snapshots, versions and held-out repositories/time periods. Compare frontier-only, constrained simple-local-first and Jev. Retain applicable EVL-001 best-single, heuristic and classifier baselines; reuse compatible frozen evidence. Count advice, embeddings, retries, escalation, caching and verification in cost per verified task. Measure quality, latency, reliability and privacy separately.

Progress from fixtures to offline/replay, authorized shadow, constrained canary and a graduation decision. Declare quality margin, latency tolerance, cost ceiling, stop criteria and rollback before paid/live runs. Preserve existing retrieval graduation gates.

Remove proven-unused role configuration, duplicate policy and obsolete documentation after coverage. Keep old reports; do not expand simulator infrastructure.

Acceptance: safety/stream tests pass; sufficient representative verified tasks support declared comparisons; no quality loss beyond agreed margin; canary kill switch is rehearsed and records its effect. No “ready” claim from an attractive average or empty hard-case cohort.

Rollback: stop canary and restore guarded native routing; keep failed evidence.

Add Codex after the Claude slice is proven

PR 7 Add Codex without forking the core

Primary ADRL-SEM-001; secondary FND-001, SEM-002/003/004, CAS-003/004, EVL-008. Depends on: proven Claude slice from PR 6.

Proposed new file: src/router/adapters/codex.py, named for the verified wire protocol rather than assuming generic OpenAI compatibility.

Changes: capture authorized current Codex protocol fixtures; implement only decoding/encoding and identity/continuity differences. Reuse advisor, constraints, execution and ledger. No copied policy or second state machine.

Acceptance: identical core contract tests plus Codex-specific tools, streaming, cancellation, utility and unknown-message cases. Run equivalent quality/cost checks before a separate Codex canary; Claude evidence alone does not establish portability.

Rollback: disable only Codex adapter; Claude remains unaffected.

Release in evidence based steps

Step	Evidence required before moving on
1 Deterministic fixtures	Protocol, constraint, retry, cancellation, ledger and failure cases pass against fixed inputs.
2 Offline and replay	Frozen tasks, snapshots and versions produce attributable results; held out repositories and time periods protect the comparison.
3 Authorized shadow	Provider and data scope, credentials, spend and concurrency are approved. Results remain observational.
4 Constrained canary	Quality, latency, cost and reliability limits are declared; stop criteria and kill switch are rehearsed.
5 Graduation decision	Representative verified evidence supports the release. Existing retrieval graduation gates remain intact.

Evaluate quality, latency, retries, reliability and privacy separately. Compare frontier only, constrained simple local first and Jev, while retaining applicable EVL-001 baselines. Include advice, embeddings, retries, escalation, cache effects and verification in cost per verified completed task.

Stop and roll back to guarded native routing when declared limits fail. Preserve failed and incomplete evidence. A favorable average or an empty hard case cohort is not a readiness claim.

Taxonomy and engineering checks

The nine buckets remain ownership categories, not nine runtime services. Each PR carries one primary ID and genuine secondary owners. All IDs use ADRL-; secondary entries abbreviate that prefix. The plan is anchored in ADRL-FND-001. It proposes contract changes but changes no repository decision state or maturity.

ID	Exact category	Main plan touchpoints
FND	System Boundary and Principles	One core process and one database; conditional fail open
SEM	Interaction Semantics	Protocol, identity and boundary rules in PRs 2 and 7
SAF	Safety, Privacy, and Hard Constraints	All outbound paths constrained in PR 1; advisor in PR 4
RTG	Routing Intelligence and Economics	One authoritative advisor choice in PR 4
CAS	Execution, Cascade, and Recovery	One execution owner and safe streaming in PR 2
MEM	Memory, Evidence, and Label Integrity	Authoritative facts in PR 3; projections in PR 5
LRN	Learning and Adaptation	Provenance, held out evidence and unchanged promotion gates
EVL	Evaluation, Graduation, and Rollout	Measured comparisons in PR 6; separate Codex proof in PR 7
OPS	Platform, Runtime, and Operations	Bounded runtime, failure handling, CI and kill switch

Checks to run during implementation

Run focused tests for each PR first, then the existing full suite. These are future checks, not executed results.

PYTHONPATH=src .venv/bin/python -m pytest tests -q

python3 tools/check_adr_taxonomy.py

Compile sources and validate readiness artifacts as required by existing CI. If ADR maturity or readiness changes, run the repository learning_readiness --persist and check_readiness_score.py workflow. Link actual evidence before advancing D0-D5 maturity.

Keep stable ADR IDs and history, or explicitly supersede rows through the index procedure. Never rewrite frozen reports or claim untested behavior as tested.

Decisions before implementation and live use

To start repository implementation

- Authorize repository execution and select the environment.
- Agree the single authoritative advisor choice representation and protocol backed continuation boundaries as part of the first contract changes.

Before any paid or live run

Choice to approve	What must be explicit
Provider and data scope	Which advisors and execution destinations may receive which context; supported credential setup.
Model candidates and fallback	Permitted candidate set, capabilities and a configured native fallback that never expands permissions.
Runtime and spend limits	Total advisor deadline, provider retry limits, concurrency and spend ceilings.
Acceptance tolerances	Quality non-inferiority margin, latency and reliability tolerances, cost ceiling and representative verification coverage.
Release controls	Shadow and canary scope, stop criteria, rollback owner and kill switch procedure.

Preparatory code and mocked tests do not require invented live choices. No calendar duration is committed; advance when the dependency and acceptance evidence for each stage is satisfied.

Reviewed sources

GitHub main reviewed at 20ca99d on 27 October 2026. Links below are pinned to that full revision. The separate architecture diagram describes the same proposed design; it is not implementation evidence.

ADR index and decision taxonomy

Repository instructions

Existing CI checks

Live routing and execution integration

Harness capture and proxy behavior

Memory contract

Memory facade and enrichment integration